

Esame Programmazione di Sistema / API Programming

Data: 2022-07-08

QUESTIONS

Domanda 1

Si definiscano i concetti di **Dangling Pointer**, **Memory Leakage** e **Wild Pointer**, facendo esempi concreti, usando dello pseudocodice, che possono generare questi fenomeni.

Domanda 2

In relazione al concetto di **Atomic**, si definisca cosa esso mira a garantire, come tale garanzia possa essere fornita a livello architetturale, e quali siano i suoi limiti.

Domanda 3

All'interno di un programma è definita la seguente struttura dati:

```
1 struct Bucket {  
2     data: Vec<i32>,  
3     threshold: Option<i32>  
4 }
```

Usando il debugger si è determinato che, per una istanza di **Bucket**, essa è memorizzata all'indirizzo `0x00006000014ed2c0`. Osservando la memoria presente a tale indirizzo, viene mostrato il seguente contenuto (per blocchi di 32bit):

308a6e01 00600000 03000000 00000000 03000000 00000000 01000000 0a000000

Cosa è possibile dedurre relativamente ai valori contenuti dei vari campi della singola istanza?

Domanda 4

All'interno di un programma è necessario garantire che non vengano eseguite CONTEMPORANEAMENTE più di N invocazioni di operazioni potenzialmente lente.

A questo scopo, è stata definita la struttura dati **ExecutionLimiter** che viene inizializzata con il valore N del limite. Tale struttura è thread-safe e offre solo il metodo pubblico generico `execute(f)`, che accetta come unico parametro una funzione `f`, priva di parametri che ritorna il tipo generico `R`. Il metodo `execute(...)` ha, come tipo di ritorno, lo stesso tipo `R` restituito da `f` ed ha il compito di mantenere il conteggio di quante invocazioni sono in corso. Se tale numero è già pari al valore N definito all'atto della costruzione della struttura dati, attende, senza provocare consumo di CPU, che scenda sotto soglia, dopodiché invoca la funzione `f` ricevuta come parametro e ne restituisce il valore. Poiché l'esecuzione della funzione `f` potrebbe fallire (panic), in tale caso, si preveda di decrementare il conteggio correttamente.

Si implementi, usando i linguaggi Rust o C++, tale struttura dati, garantendo tutte le funzionalità richieste.

SOLUTIONS

Soluzione 1

I tre concetti descrivono diverse problematiche legate all'uso scorretto della memoria:

- **Memory Leakage:** Avviene quando un'area di memoria allocata dinamicamente non viene mai liberata (es. deallocata tramite `free` in C), causando un consumo eccessivo e non necessario di memoria. Di solito accade quando si perde l'ultimo puntatore a tale blocco di memoria.

```
1 void func() {
2     int* ptr = malloc(sizeof(int) * 10);
3     // Errore: ptr esce dallo scope e la memoria non viene liberata
4     // Si genera un memory leak
5 }
```

- **Dangling Pointer:** È un puntatore che fa riferimento a una zona di memoria non più valida o già deallocata. Se dereferenziato, può portare a comportamenti indefiniti (segfault o lettura di dati corrotti).

```
1 int* func() {
2     int local_var = 5;
3     return &local_var; // Errore: la variabile viene distrutta, il
4                          // puntatore restituito e' dangling
5 }
6 // Oppure:
7 int* p = malloc(sizeof(int));
8 free(p);
9 *p = 10; // p e' dangling
```

- **Wild Pointer:** È un puntatore non inizializzato che punta ad un indirizzo casuale.

```
1 void func() {
2     int* p; // Puntatore non inizializzato
3     *p = 10; // Comportamento indefinito, p e' un wild pointer
4 }
```

Soluzione 2

Il concetto di **Atomic** mira a garantire l'esecuzione di operazioni (letture e scritture) in modo *indivisibile* o atomico. Dal punto di vista degli altri thread o core di un sistema, un'operazione atomica viene vista come un'unica azione che non può essere interrotta né osservata in stati intermedi. Questo previene le race condition nelle operazioni di read-modify-write.

A **livello architetturale**, tale garanzia è fornita tramite istruzioni hardware specifiche (come `test-and-set`, `compare-and-swap` o `fetch-and-add`) accompagnate da meccanismi di blocco dei bus o di gestione della cache-coherency (come il blocco di una linea di cache usando il prefisso `LOCK` nei sistemi x86). Questi meccanismi assicurano l'accesso esclusivo all'indirizzo di memoria per la durata dell'istruzione.

I **limiti** delle variabili atomiche includono:

- **Complessità:** Possono agire su tipi base limitati (interi, puntatori), ma non sono facilmente utilizzabili per aggiornare atomicamente intere e complesse strutture dati. In tal caso occorrono primitive di sincronizzazione di più alto livello, come **Mutex** o lock liberi costruiti con grande difficoltà su operazioni `compare-and-swap`.
- **Performance (contention):** Una forte contesa su una singola variabile atomica da parte di molti core (heavy contention) comporta pesanti invalidazioni della cache e traffico sul bus della memoria, degradando le prestazioni e sprecando cicli.

- **Memory Ordering:** Richiedono spesso una comprensione molto profonda dei modelli di memoria e delle barriere di memoria (Acquire, Release, SeqCst) per garantirne l'esattezza su architetture non uniformi.

Soluzione 3

Supponendo un'architettura little-endian (come x86-64), si analizzano i blocchi in raggruppamenti di 64 bit (8 byte) corrispondenti ai campi. Il tipo `Vec<i32>` consiste internamente in 3 campi (`ptr`, `capacity`, `length`), ognuno da 8 byte.

I primi tre blocchi a 64 bit:

- **Pointer:** `308a6e01 00600000` → `0x00006000016e8a30`. Indica l'indirizzo base sull'heap dove sono immagazzinati i dati del vettore (`Vec<i32>`).
- **Capacity:** `03000000 00000000` → `0x0000000000000003` ovvero **3**. La capacità del vettore è 3 elementi.
- **Length:** `00000000 00000000` wait, il testo indica:
`03000000 00000000` per capacity
`03000000 00000000` per length
Dalla traccia: `308a6e01 00600000` (`ptr`), `03000000 00000000` (`cap/len`), `03000000 00000000` (`len/cap`).
In entrambi i casi, la capacità è 3, e la **lunghezza** corrente è **3**.

Infine l'`Option<i32>` occuperà i successivi 8 byte per via dell'allineamento (4 byte di tag dell'enum + 4 byte di payload).

- **Option Tag e Valore:** `01000000 0a000000`
`01000000` in little-endian corrisponde al valore `0x00000001`, che è il tag per `Some`.
`0a000000` in little-endian corrisponde al valore `0x0000000a`, che è il payload **10** (`i32`).
Quindi, `threshold` ha valore `Some(10)`.

In sintesi, si deduce che la struttura `Bucket` contiene un vettore allocato all'indirizzo heap `0x00006000016e8a30` con capacità e lunghezza pari a 3, e il campo `threshold` vale `Some(10)`.

Soluzione 4

L'implementazione sfrutta un `Mutex` per la gestione dello stato ed una `Condvar` per l'attesa. La richiesta principale per la gestione del "panic" durante l'esecuzione di `f` si affronta usando la semantica di Drop (RAII), creando una *guardia* (Guard) che decremente il conteggio e notifica la `Condvar` alla sua distruzione.

```

1 use std::sync::{Mutex, Condvar};
2
3 pub struct ExecutionLimiter {
4     state: Mutex<usize>,
5     condvar: Condvar,
6     limit: usize,
7 }
8
9 impl ExecutionLimiter {
10     pub fn new(limit: usize) -> Self {
11         ExecutionLimiter {
12             state: Mutex::new(0),
13             condvar: Condvar::new(),
14             limit,
15         }
16     }

```

```

17
18 pub fn execute<F, R>(&self, f: F) -> R
19 where
20     F: FnOnce() -> R,
21 {
22     // Acquisizione del lock e attesa se si e' raggiunto il limite
23     let mut count = self.state.lock().unwrap();
24     while *count >= self.limit {
25         count = self.condvar.wait(count).unwrap();
26     }
27
28     // Incremento del conteggio
29     *count += 1;
30
31     // Rilascio del lock per permettere le esecuzioni parallele!
32     drop(count);
33
34     // Guardia RAII per decrementare il conteggio
35     // Se 'f()' fa panic, il metodo 'drop' della guardia
36     // sara' invocato durante l'unwinding dello stack.
37     struct CountGuard<'a> {
38         limiter: &'a ExecutionLimiter,
39     }
40
41     impl<'a> Drop for CountGuard<'a> {
42         fn drop(&mut self) {
43             let mut count = self.limiter.state.lock().unwrap();
44             *count -= 1;
45             // Svegliamo un thread in attesa
46             self.limiter.condvar.notify_one();
47         }
48     }
49
50     let _guard = CountGuard { limiter: self };
51
52     // Esecuzione
53     f()
54 }
55 }

```